Cyprus University of Technology

CSE 226
Algorithms and Complexity

Lecture 3
Divide and Conquer
Διαίρει και Κυρίευε

Herodotos Herodotou

Contains edited slides from T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, D. Luebke, and M. Christodoulakis

1

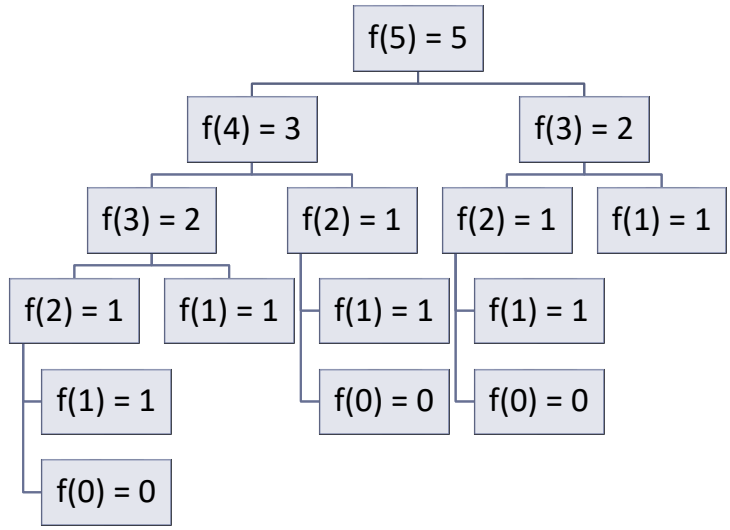
Recursive Algorithms

- An algorithm is **recursive** (**αναδρομικός**) when, in order to solve a problem, it calls itself recursively one or more times
- Example
  - Fibonacci numbers are recursively defined as $f_n = f_{n-1} + f_{n-2}$, where $n \geq 2$ and $f_0 = 0$ and $f_1 = 1$
  - Recursive algorithm to compute the nth Fibonacci number:

```
fibonacci(n){
   if n=0 then return 0;
   if n=1 then return 1;
   return fibonacci(n-1) + fibonacci(n-2);
}
```


2

Recursive Algorithms – Fibonacci



Divide and Conquer

- Many recursive algorithms follow the **divide-and-conquer** (**διαίρει-και-κυρίευε** ή **διαίρει-και-βασίλευε**) approach
- It consists of three steps
  - **Divide** (**Διαιρούμε**) the problem into a number of subproblems
  - **Conquer** (**Κυριεύουμε**) the subproblems by solving them recursively
    - Note: When the subproblem size becomes small enough, just solve it in a straightforward manner!
  - **Combine** (**Συνδυάζουμε**) the solutions to the subproblems into the solution for the original problem
- The **MergeSort** (**Συγχωνευτική Ταξινόμηση**) algorithm is a divide-and-conquer algorithm

MergeSort – Idea

- Divide and conquer in MergeSort
  - **Divide:** Divide the n-element sequence to be sorted into two subsequences of $n/2$ elements each
  - **Conquer:** Sort the two subsequences recursively using merge sort
  - **Combine:** Merge the two sorted subsequences to produce the sorted answer
- *Question*: what is the smallest-size subproblem that we will solve straightforwardly?
- *Answer:* $n = 1$, because in that case the list is already sorted (nothing to be done)

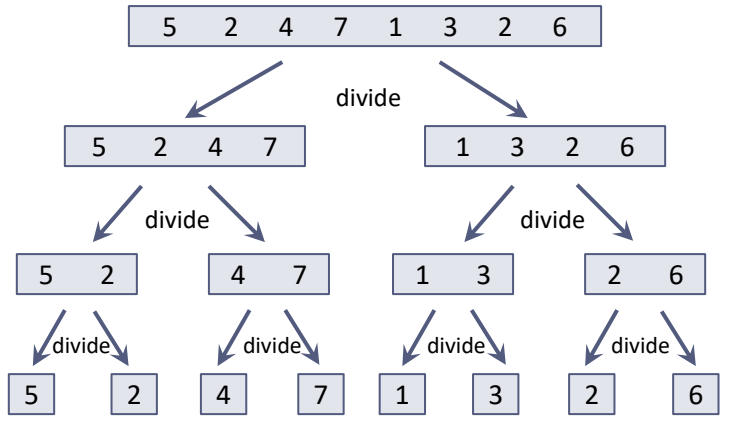
MergeSort – Algorithm

```
MergeSort(A, left, right) {
   if left >= right
      return
   mid = floor((left + right) / 2)
   MergeSort(A, left, mid)
   MergeSort(A, mid+1, right)
   Merge(A, left, mid, right)
}
```

- Note: **Merge** is another algorithm used for merging the two sorted lists. It performs the third step of the divide and conquer technique (the "*combine*" step)

MergeSort – Example

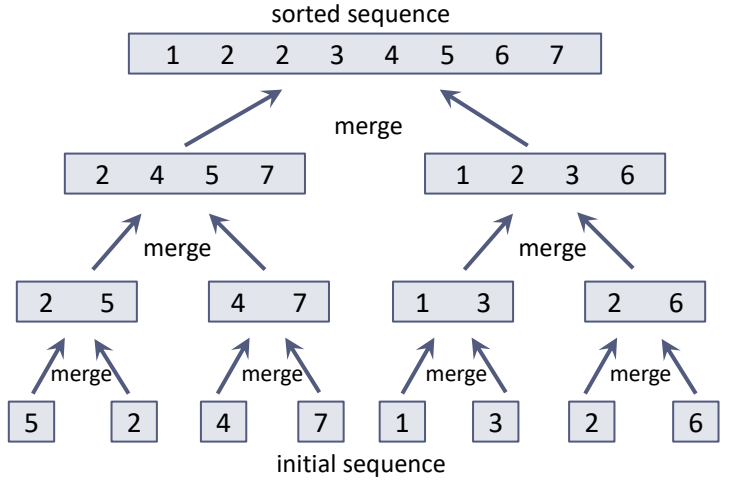
- MergeSort on input A = (5,2,4,7,1,3,2,6)



7

MergeSort – Example

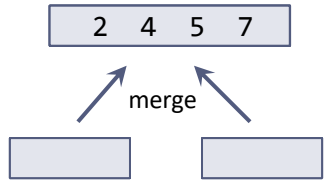
- MergeSort on input A = (5,2,4,7,1,3,2,6)



8

MergeSort – The merge step

- Notice that the lists that are input to **`Merge`** are sorted!
- Idea
  - Create pointers, i and j, to point at the beginning of each list
  - Compare the elements at the pointers
  - Write the smallest of the two and move the corresponding pointer one position to the write
  - When one of the lists is exhausted, simply copy the remaining elements of the other list at the end of the output



Merge Pseudocode

```
Merge(A, left, mid, right) {
   n = right-left+1
   sorted[1..n] = [0..0]
   i = left, j = mid+1, k = 1
   while (i<=mid OR j<=right) {
      if ((i<=mid AND j<=right AND A[i]<=A[j]) OR j>right) {
         sorted[k] = A[i]
         i = i+1, k = k+1
      } else {
         sorted[k] = A[j]
         j = j+1, k = k+1
      }
   }
   A[left..right] = sorted[1..n]
}
```

- *What is the (asymptotic) running time of the algorithm?*

Analysing divide-and-conquer algorithms

- When an algorithm contains a recursive call to itself, its running time can often be described by a **recurrence equation** (**αναδρομική εξίσωση**)
- Notation
  - Let $T(n)$ denote the running time at input size n
  - Let c denote the (usually small) input size for which a straightforward solution exists
  - Suppose that the problem is divided into a subproblems, each of which is $1/b$ the size of the original
  - Let $D(n)$ denote the time to divide the problem and $C(n)$ the time to combine the solutions
- The running time will be

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq c \\ aT(n/b) + D(n) + C(n) & otherwise \end{cases}$$

MergeSort – Analysis

- Assume, for the sake of simplicity, that n is a power of 2
- We divide the problem to 2 subproblems, half the size of the initial one, hence $a = 2$ and $b = 2$
- When the list size is 1, the problem is solved trivially ($c = 1$)
- Time spent at each step of the divide-and-conquer
  - **Divide**: we just find the middle of the subarray, $D(n) = \Theta(1)$
  - **Conquer**: we solve two subproblems of size $n/2$, i.e., $2T(n/2)$ time
  - **Combine**: merge the lists with Merge, so $C(n) = \Theta(n)$
- Hence, the recursive equation is

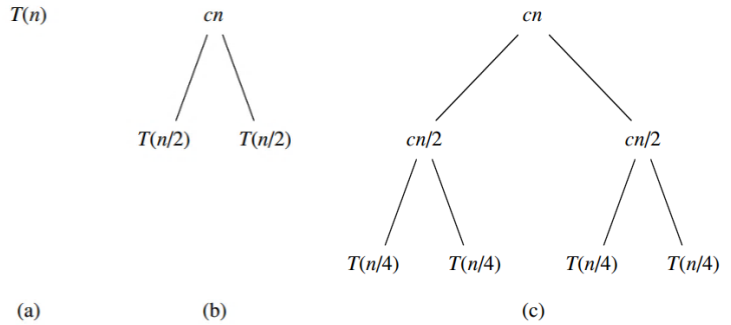
$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$

Intuitively solving the recurrence equation

- Let's re-write the previous equation as follows

$$T(n) = \begin{cases} c & \text{if } n = 1 \\ 2T(n/2) + cn & \text{if } n > 1 \end{cases}$$

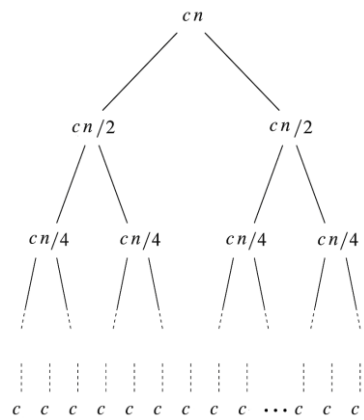
- The **recursion tree** (**δένδρο αναδρομής**) at the first steps:



Intuitively solving the recurrence equation

Observations:

- The ith level has 2^i subproblems
- Each subproblem has cost $cn/2^i$
- At the leafs
  $cn/2^i = c$
  $\Rightarrow n = 2^i$
  $\Rightarrow i = \lg n$



Intuitively solving the recurrence equation

- In total, the tree looks like this
- Notice that we have $\lg n + 1$ levels
- By adding up the costs of each level: $cn \lg n + cn$
- Thus, $\boldsymbol{\Theta(n \lg n)}$

